SCA101 - LAB 2_1B (EXPLORER)

Password Bypass Explorer

Step-by-step student guide: edit the numbers, watch the attack work

In Lab 2_1B you recover a password from a chip's power trace. This guide goes with a **second, hands-on notebook** where you change the numbers yourself - which guesses to compare, how to score them, how many captures to average, how much noise to add - and watch the results update, so you can see **why** the attack works and what would stop it. No hardware is needed: it runs on recorded traces, or on your own chip if you have one.

> **HOW TO USE THIS GUIDE**
> This guide goes **with** the notebook - it does not replace it. Keep the notebook open next to this PDF. Every step says what to edit, what you should see, and what to try if you do not see it.

Which files are which

File (folder: jupyter/courses/sca101/)	What it is	Open it?
`Lab 2_1B - Password Bypass Explorer.ipynb`	The Explorer. Everything in this guide happens here.	**Yes**
`Lab 2_1B - Power Analysis for Password Bypass (MAIN).ipynb`	The original lab. Do it before or after the Explorer.	Optional
`SOLN_Lab 2_1B - ...ipynb`	Answer key for the original lab.	If stuck
`Lab 2_1B - Firmware Change and Flash Walkthrough.ipynb`	How to change the password on your own chip (hardware).	Optional
`... (HARDWARE).ipynb`, `... (SIMULATED).ipynb`	Back-end helpers that the notebooks run for you.	**No**

The four colored boxes you will see everywhere

DO THIS
An action to take.

YOU SHOULD SEE
What the screen should show.

IF IT DOESN'T
What to try if it didn't.

WATCH OUT
A mistake that costs time.

Guide version 1.0 - November 2026. The numbers in this guide come from running the notebook on **Ubuntu 22.04** with the recorded traces; the simulator picks a random recorded trace per guess, so your last digits will differ. Step 7 (your own chip) has not been run on a live Husky for this guide.

Contents

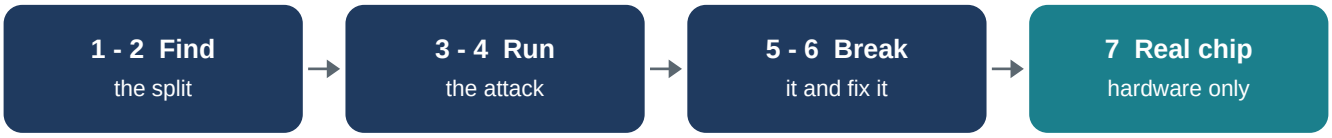
The big picture . . . 3
Before you start . . . 4
How every edit-and-run cell works . . . 4
Setup - open the notebook and run it . . . 5
S.1 Open the Explorer . . . 5
S.2 Choose the version . . . 5
S.3 Run the two Setup cells . . . 5
S.4 Optional: run everything once . . . 5
Steps 1 and 2 - find the split . . . 6
1 Look at a guess . . . 6
2 Where do a right and a wrong guess split? . . . 7
Steps 3 and 4 - score the characters, run the attack . . . 8
3 Score every possible character . . . 8
4 Run the whole attack . . . 9
Steps 5 and 6 - break it, then fix it . . . 10
5 What breaks the attack? Noise - and what fixes it . . . 10
6 A defence: random delays . . . 12
Step 7 - run it on your own chip (hardware) . . . 13
Step 8 - check your understanding . . . 14
You are done when... . . . 15
Troubleshooting . . . 16
Setup and the recorded traces . . . 16
Results that look wrong . . . 16
Hardware (Step 7) . . . 17
Appendix A - every variable you can edit . . . 18
Appendix B - Glossary . . . 19

The big picture

What is the attack? A simple password checker compares your guess with the real password one character at a time and **stops at the first wrong character**. A guess whose first character is right therefore gets one comparison further than a wrong one - it runs different code for longer - and that difference shows up in the chip's **power trace**. By finding which character makes the trace change, you recover the password **one character at a time** without ever being told it.

What is different about the Explorer? The original lab walks you through the whole attack once. The Explorer lets you **poke at every part of it**: every code cell has a clearly marked block of numbers you can change. Change one, press **Shift+Enter**, and the plots and the printed numbers change straight away.

Your path through the notebook:



Step	What you do	What you learn	Hardware?
1	Overlay a right and a wrong guess.	A right character makes the trace take a different path.	No
2	Find the sample where the traces split.	The split moves **40 samples later** for every correct character.	No
3	Score every possible character.	The right one scores about 30x higher; how window and scoring method change that.	No
4	Run the whole attack.	185 captures instead of 60 million guesses.	No
5	Add noise; average captures.	How much noise the attack can take - and that averaging cuts noise by sqrt(R).	No
6	Add random delays.	A few samples of delay break a single capture; averaging restores it, up to a point.	No
7	Run it on your own chip.	The same code works on real traces.	**Yes**
8	Answer seven questions.	Put it all together.	No

Before you start

Tick every row. Most problems later come from something on this list.

	You need	How to check
[ ]	**Lab 0 - SCA101 Setup** passed.	ChipWhisperer, numpy and matplotlib import without errors.
[ ]	The `cwtraces` package installed. It holds the recorded traces for the simulated labs.	`python -c "import cwtraces"` prints nothing. If it says `ModuleNotFoundError`, run `python -m pip install cwtraces`.
[ ]	Jupyter running from your ChipWhisperer environment.	On Linux see the Linux guide Part 3.
[ ]	The course notebooks, including the Explorer.	`jupyter/courses/sca101/` contains `Lab 2_1B - Password Bypass Explorer.ipynb`.
[ ]	**Only for Step 7:** the Husky kit assembled and plugged in.	Lab 0's hardware test ends with `hardware OK (disconnected again)`.

WATCH OUT

`cwtraces` **is not installed by the standard install steps.** `jupyter/requirements.txt` does not list it, so on a fresh install the simulated Lab 2_1B (and this Explorer) stops with `ModuleNotFoundError: No module named 'cwtraces'`. `python -m pip install cwtraces` fixes it (on Ubuntu 22.04 it installed version 0.2.0 and left numpy unchanged).

GOOD TO KNOW

You cannot break anything. The simulator only ever *returns* recorded traces. If a plot looks wrong or an error appears, use **Kernel > Restart & Run All** and you are back to the starting state.

How every edit-and-run cell works

Do this	Keys
Run the cell and move to the next one	**Shift + Enter**
Run the cell and stay on it (best for repeated experiments)	**Ctrl + Enter**
Start over from the top	**Kernel > Restart & Run All**

Every cell you are meant to change starts with the line `# EDIT THESE`. Change only what is in that block, then run the cell. Everything below it just does the drawing and the maths. Steps 5 and 6 need Step 5's cell to have been run first.

Setup - open the notebook and run it

S.1 Open the Explorer

1. Start Jupyter (for Linux: `source ~/.cwvenv/bin/activate`, then `cd ~/chipwhisperer/jupyter`, then `jupyter notebook`).
2. Click **courses**, then **sca101**.
3. Click **Lab 2_1B - Password Bypass Explorer.ipynb**.

S.2 Choose the version

Variable	Default	What it does
`VERSION`	`'SIMULATED'`	`'SIMULATED'` = recorded traces from the `cwtraces` package (no hardware). `'HARDWARE'` = your own Husky + SAM4S: the Setup cell builds and flashes the firmware for you.

S.3 Run the two Setup cells

YOU SHOULD SEE

The second cell prints a green check mark and `Ready! Mode: SIMULATED | trace length: 3000`. It defines the helper functions used by every later step, including `get_trace(guess)` - send a guess, get its 3000-sample power trace.

IF IT DOESN'T

`ImportError: The 'cwtraces' package is missing ...` - install it as described above, restart the kernel (E-1). `ValueError: VERSION must be ...` - mistyped; it must be exactly `'SIMULATED'` or `'HARDWARE'`.

S.4 Optional: run everything once

Choose **Kernel > Restart & Run All** to see the finished notebook. In simulated mode it takes about a minute (Step 4 makes 185 captures; Steps 5 and 6 run many attacks) and should finish with no red error boxes.

Steps 1 and 2 - find the split

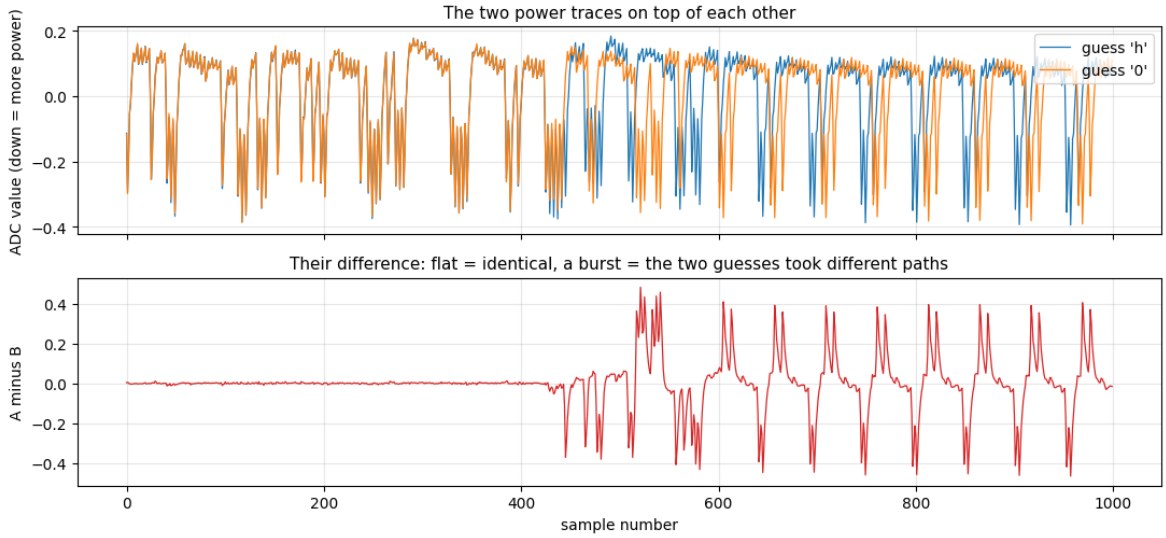
1 Look at a guess

The checker compares your guess with the stored password with a loop that **stops at the first mismatch**:

```
for(uint8_t i = 0; i < sizeof(correct_passwd); i++){
    if (correct_passwd[i] != passwd[i]){
        passbad = 1;
        break;                          // stop at the first mismatch
    }
}
```

The ChipWhisperer records **3000 samples** while this runs. As in Lab 2_1A the trace is **upside down**: bigger negative swings mean more power. The notebook adds the `\n` that the serial protocol needs, so you type guesses without it.

Variable	Default	What it does
`GUESS_A`, `GUESS_B`	`"h"`, `"0"`	The two guesses to compare (without the newline).
`START, END`	`0, 1000`	Zoom window in samples (the whole trace is 0 to 3000).
`SMOOTH`	`1`	Smoothing width: 1 = raw samples, bigger = smoother.



A right first character (`h`) against a wrong one (`0`): identical at first, then different from about sample 430.

YOU SHOULD SEE

The top plot shows two traces that sit on top of each other until about sample 430, then drift apart. The bottom plot - their difference - is **flat until about sample 430**, then bursts. The last line prints `total difference between the two traces: 302.7` (yours will be close to 300).

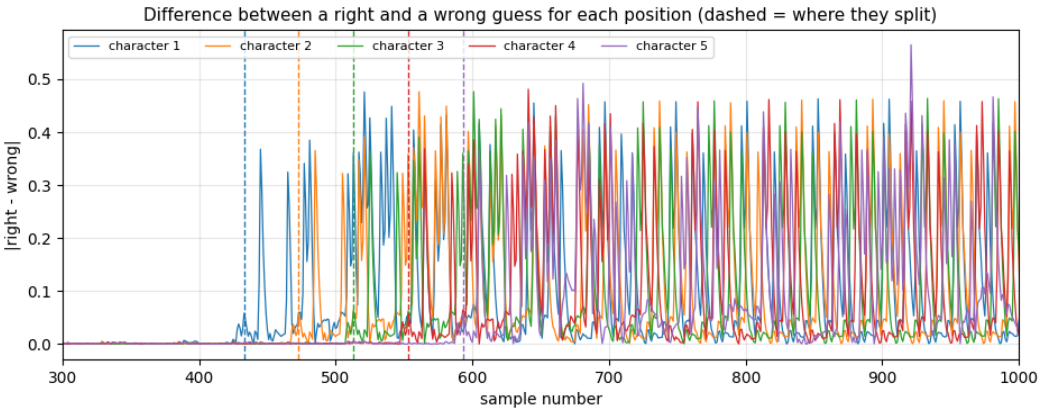
DO THIS

Try: (1) `GUESS_A = "h"`, `GUESS_B = "x"`: another wrong character - is the picture the same? (2) `"h0"` against `"hx"`: now the first character is right for both; where do they split? (3) `"zzzzz"` against `"azzzz"`. (4) Raise `SMOOTH` to 9.

2 Where do a right and a wrong guess split?

To understand the leak, be the lab designer: we **know** the password (the simulator's is `h0px3`), so we can compare the **right** first `k+1` characters with the first `k` characters followed by `\x01` (a byte that is never part of a password), for each position `k`, and find the exact sample where the traces stop matching.

Variable	Default	What it does
`STUDY_PASSWORD`	`"h0px3"`	The password to study. On your own chip, use whatever you flashed.
`THRESHOLD`	`0.05`	A difference bigger than this counts as 'the traces have split'.
`AVERAGES`	`10`	Captures averaged for each trace (calms the noise).
`SMOOTH_DIFF`	`1`	Smoothing applied to the difference before looking for the split.



Difference between a right and a wrong guess for each position; the dashed lines mark where they split.

YOU SHOULD SEE

A table of five rows. The traces split at samples **433, 473, 513, 553 and 593**, and the last column shows **40** every time: **each extra correct character pushes the split 40 samples later.** That is the time the checker spends on one more comparison.

GOOD TO KNOW

How long is 40 samples? The time per character is fixed, but how many samples that covers depends on the capture settings. In Lab 2_1A the Husky took 4 samples per clock cycle, which would make 40 samples **10 clock cycles**. The recorded traces here come from a different capture setup (their autocorrelation shows no clear clock pattern), so the notebook counts in samples.

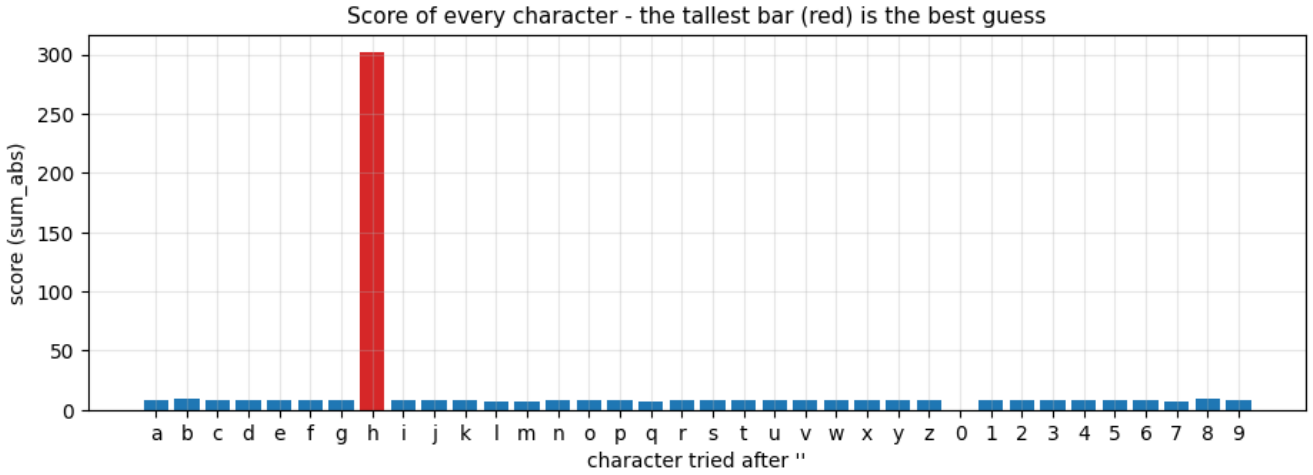
What we changed	Result
`THRESHOLD = 0.2`	Splits at 445, 485, ... (still 40 apart, but the 'first big difference' comes a little later)
`THRESHOLD = 0.01`	Splits at 428, 468, ... (earlier: even tiny differences count)
`AVERAGES = 1`	433, 473, ... - the same. The recorded traces are very clean.

Steps 3 and 4 - score the characters, run the attack

3 Score every possible character

Now the real attack: we **don't** know the password. Try every possible character after what we know so far, compare each trace with a wrong-guess reference (`\x01`), and give each a **score** - how different it is from the reference. The right character takes the other path, so it scores far higher than the 35 wrong ones.

Variable	Default	What it does
`KNOWN_PREFIX`	`""`	The characters already found (start with nothing).
`WINDOW_START, WINDOW_END`	`0, 3000`	Which samples to compare; the rest are ignored.
`METRIC`	`"sum_abs"`	How to turn the difference into one number: `sum_abs` (add up sizes), `sum_sq` (add up squares) or `max_abs` (the single biggest).
`REPEATS`	`1`	Captures averaged per guess.



Score of every first character: h towers over the other 35.

YOU SHOULD SEE

One red bar far taller than the rest, and a line such as `best character: 'h' (score 301.5) | runner-up: 'q' (8.8) | margin: 34.1x`.

What we got when we changed the settings:

Setting	Best character (score)	Runner-up	Margin	Right answer?
defaults: window 0-3000, `sum_abs`	`h` (301.5)	`q` (8.8)	34x	yes
window 440-600	`h` (21.2)	`7` (0.6)	36x	yes
window 400-500	`h` (5.5)	`3` (0.4)	15x	yes
window 0-300	`x` (1.5)	`a` (1.5)	1.0x	**no** - a tie
`METRIC = "sum_sq"`	`h` (76.4)	`l` (0.1)	1364x	yes
`METRIC = "max_abs"`	`h` (0.5)	`d` (0.0)	16x	yes
`KNOWN_PREFIX = "h0p"`	`x` (287.2)	`o` (8.9)	32x	yes (the 4th character)

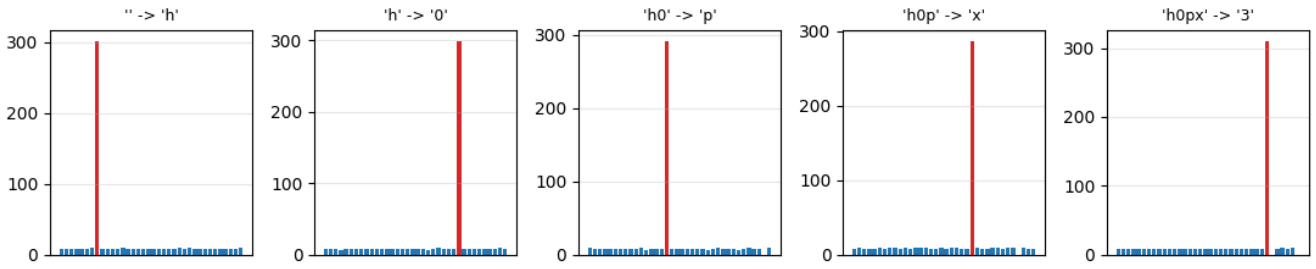
DO THIS

Try: (1) Walk the prefix: `"h"`, `"h0"`, `"h0p"` - you are finding characters 2, 3, 4. (2) Window `0, 300`: why does it fail? *(The first split is at sample 433 - the window ends before the traces differ.)* (3) Compare the margins for the three metrics - which separates the right character most clearly? (4) `REPEATS = 4`: does the margin change?

4 Run the whole attack

Repeat Step 3 for position 1, then 2, then 3... each time adding the best-scoring character to the password found so far. That is **five rounds of 36 guesses** instead of trying every possible 5-character password.

Variable	Default	What it does
`PASSWORD_LENGTH`	`5`	How many characters the password has.
`ATTACK_START`, `ATTACK_END`	`0, 3000`	Which samples to compare.
`ATTACK_METRIC`	`"sum_abs"`	`sum_abs`, `sum_sq` or `max_abs`.
`ATTACK_REPEATS`	`1`	Captures averaged per guess.



One bar chart per round: in every round exactly one character towers over the other 35.

YOU SHOULD SEE

`round 1: 'h'`, `round 2: 'h0'`, ... `round 5: 'h0px3'`, then `recovered password: 'h0px3'`, `captures used: 185 (in 16 s)`, and `brute force would need up to 60,466,176 guesses; the power attack used 185 - about 326,844 times fewer.` In simulated mode the last line also says `MATCH` against the simulator's hidden password.

What we changed	Recovered	Captures
`ATTACK_END = 600`	`h0px7` - four right	185
`ATTACK_END = 500`	`h0lbl` - first two right	185
`ATTACK_END = 480`	`h0gud` - first two right	185
`ATTACK_METRIC = "sum_sq"` **or** `"max_abs"`	`h0px3`	185
`ATTACK_REPEATS = 2`	`h0px3`	370 (about 28 s)

DO THIS

Try: set `ATTACK_END` to 500, 480 and 600 as in the table. The window must reach **past the split point**, which moves 40 samples later for every correct character (433, 473, 513, 553, 593): with the window ending at 500 the third character's split (513) is outside it, so every candidate scores about the same and the attack picks one at random. Which characters does each setting get right?

Steps 5 and 6 - break it, then fix it

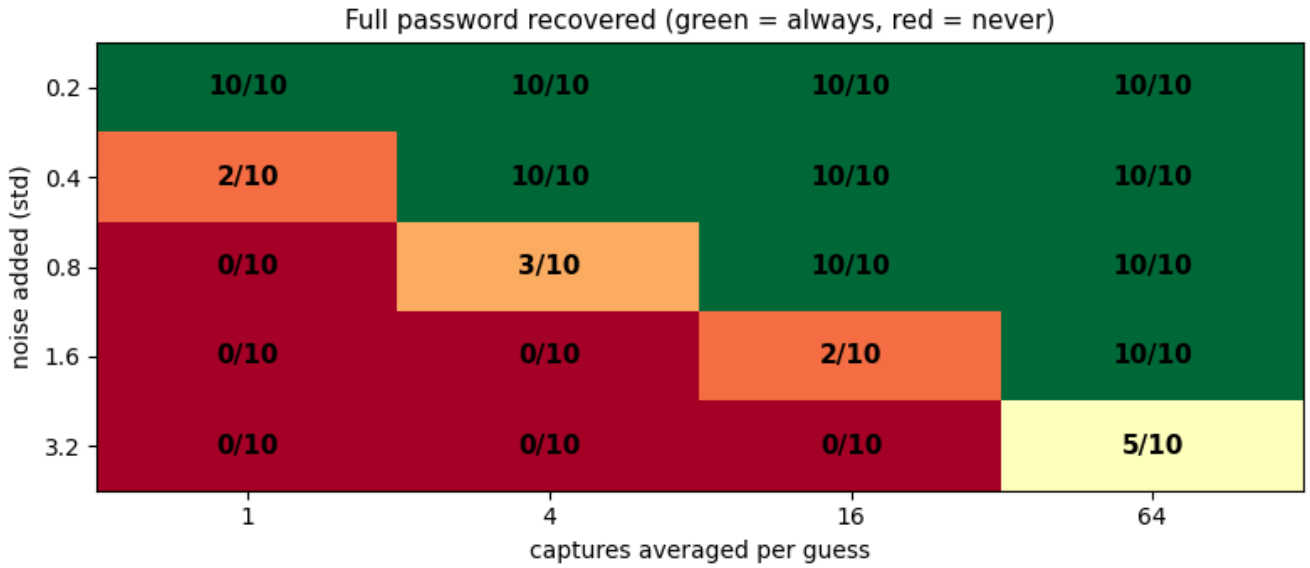
5 What breaks the attack? Noise - and what fixes it

The recorded traces are extremely clean. A real chip next to a noisy supply is not. To **turn the noise up** and run the attack many times quickly, the notebook builds a **model from real traces**: for each position it captures one clean 'right' and one clean 'wrong' trace (averaged), then adds random noise to every simulated capture. Averaging `R` captures reduces the noise by a factor of the square root of `R`, so the model uses a smaller noise for repeats. Each cell of the grid runs the whole attack `TRIALS` times and counts how often it finds the full password.

WATCH OUT

This is a model built from real data, not a new measurement. It answers 'how much noise can the attack take?', not 'what are my chip's exact numbers?'.

Variable	Default	What it does
`NOISE_LEVELS`	`[0.2, 0.4, 0.8, 1.6, 3.2]`	Noise (standard deviation) added to every sample. The real recordings have about 0.003.
`REPEAT_LEVELS`	`[1, 4, 16, 64]`	Captures averaged per guess.
`TRIALS`	`10`	Full attacks per cell.
`MODEL_WINDOW`, `MODEL_METRIC`	`(0, 3000)`, `"sum_abs"`	Samples compared and scoring method.



Attacks that recovered the whole password (out of 10): green = always, red = never.

What we got (one run of the defaults - yours will differ a little because the noise is random):

noise \ captures averaged	1	4	16	64
0.2	10/10	10/10	10/10	10/10
0.4	2/10	10/10	10/10	10/10
0.8	0/10	3/10	10/10	10/10
1.6	0/10	0/10	2/10	10/10
3.2	0/10	0/10	0/10	4/10

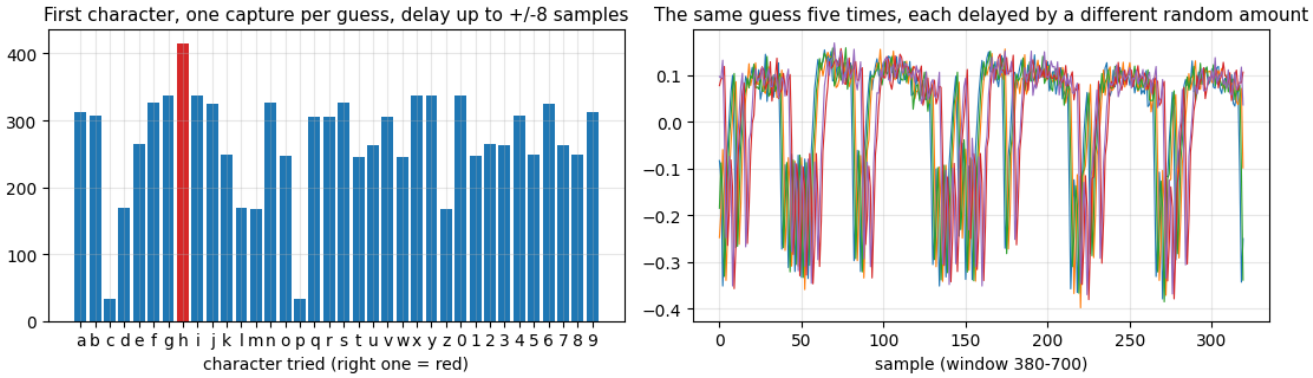
DO THIS

Try: (1) Read along the first column: where does a single capture stop working? (2) Read down a column: each time you quadruple the averaging, how much more noise can the attack take? (About double.) (3) `MODEL_WINDOW = (0, 1000)`: the attack tolerates **less** noise (at noise 0.8 with 4 captures it drops to 0/10, and at 3.2 with 64 captures from 4/10 to 0/10) - a bigger window collects more signal.

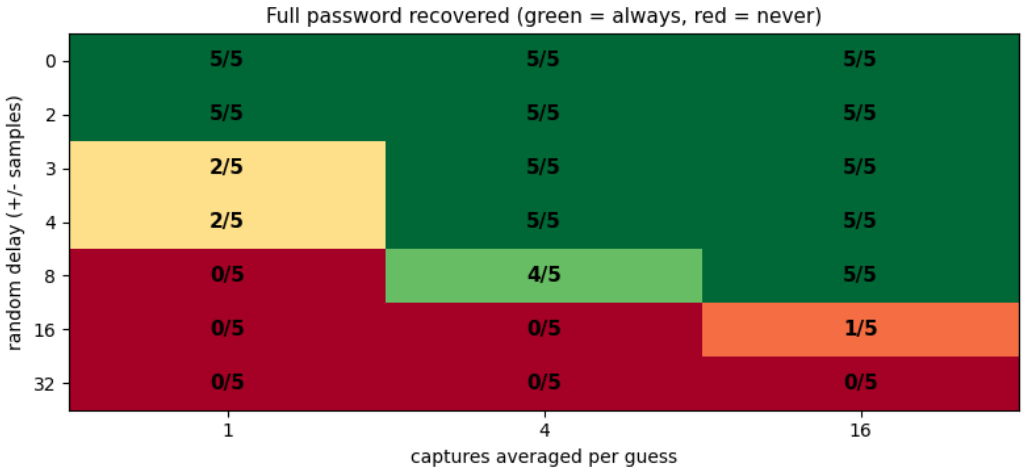
6 A defence: random delays

A well-known defence is to make the chip wait a **random** time before it does the check. The same guess then gives a trace **shifted** by a different amount every time. Our scoring compares samples at the *same position*, so even two identical guesses look different. The cell shifts every simulated capture by a random number of samples between `-JITTER` and `+JITTER` and, like Step 5, runs the whole attack several times for each combination of delay size and captures averaged.

Variable	Default	What it does
`JITTER_LEVELS`	`[0, 2, 3, 4, 8, 16, 32]`	Random delay sizes (+/- samples) in the grid.
`JITTER_REPEATS`	`[1, 4, 16]`	Captures averaged per guess.
`JITTER_NOISE`, `JITTER_TRIALS`	`0.01`, `5`	A little ordinary noise; full attacks per cell.
`SHOW_JITTER`	`8`	The delay used for the two example charts.



Left: with a delay of up to +/-8 samples the wrong characters score almost as high as the right one. Right: five copies of the same guess no longer line up.



Attacks that recovered the whole password: delay size (rows) against captures averaged (columns).

What we got (6 attacks per cell):

delay (+/- samples) \ captures averaged	1	4	16
0	6/6	6/6	6/6
2	6/6	6/6	6/6
3	1/6	6/6	6/6
4	2/6	6/6	6/6
8	0/6	3/6	6/6
16	0/6	0/6	0/6
32	0/6	0/6	0/6

YOU SHOULD SEE

A single capture survives a delay of 2 samples but breaks at **3**. Averaging **4 to 16** captures restores the attack up to about **8** samples. At **16 samples or more** (bigger than the features in the trace) even 16 averages barely help.

DO THIS

Try: (1) Explore the edge: `JITTER_LEVELS = [3, 4, 5, 6, 7, 8]`. (2) Why does averaging help at all, when every capture is shifted differently? *(The shifts are random and independent, so the average is a smooth blur in which the right guess still stands out - like noise.)* (3) What would an attacker do about a 32-sample delay? *(Re-align each trace before comparing - the subject of later labs.)*

Step 7 - run it on your own chip (hardware)

Everything except the models in Steps 5 and 6 works unchanged on the real chip, because both back-ends give you the same `cap_pass_trace()` function. (Steps 5 and 6 build their model from your **live** traces, so they work too, just with a slower start.)

DO THIS

Before you start: (1) Assemble the Husky kit as in Lab 0 and plug it in; check the **SAM4S** target board is the one mounted. (2) In the first cell set `VERSION = 'HARDWARE'`. (3) Choose **Kernel > Restart & Run All** - the Setup cell builds and flashes `basic-passwdcheck`. (4) Close any other notebook connected to the Husky: only one program can hold it.

YOU SHOULD SEE

The Setup cell finishes with `Ready! Mode: HARDWARE | trace length: 3000`. The Step 7 cell then times one capture and estimates how long Step 4 will take (185 captures). Step 4 on real hardware takes a minute or two.

Experiments - predict first, then run:

Change	What do you predict?	How you will know
Step 2 on your chip	Is the split-to-split gap still 40 samples?	The `later than the previous by` column.
Step 3 with `REPEATS = 1`, then `4`	Is your chip noisier than the recorded traces?	The margin printed under the bar chart.
Change the password in the firmware (see the Firmware Change and Flash Walkthrough notebook); set `STUDY_PASSWORD` and `PASSWORD_LENGTH` to match; run Step 4	Does the attack find your new password?	Step 4's recovered password.

> **WATCH OUT**
>
> **On hardware the firmware's real password is the answer.** The line that says `MATCH` against `h0px3` only appears in simulated mode; if you changed the firmware password, compare the recovered one with your own.

Step 8 - check your understanding

Try each question first. The answers are underneath - cover them up!

#	Question	Answer
1	Why does a right first character look different from a wrong one in the power trace?	The checker stops at the first wrong character. With a right character it carries on to compare the next one, so it runs **different code for longer** - which draws a different amount of power over time.
2	How much later does the split happen for each extra correct character?	**40 samples** (samples 433, 473, 513, 553, 593). That is the time of one more comparison; if your capture takes 4 samples per clock cycle (as the Husky's default setup does in Lab 2_1A) it is 10 clock cycles.
3	Why did the attack with `ATTACK_END = 500` get the first two characters but fail after that?	The window stops at sample 500, but the third character's split is at about 513 and later ones even further. Outside the window the traces look identical, so every wrong character scores about the same and the attack picks one at random. **The window must reach past the split point, which moves later with each correct character.**
4	How many guesses does brute force need for a 5-character password from 36 characters? How many captures did the power attack use?	**36^5 = 60,466,176** in the worst case. The power attack used **185** (5 rounds of 36 guesses plus 5 references) - hundreds of thousands of times fewer.
5	Why does averaging more captures rescue the attack when the noise goes up? What does it cost?	Noise adds random values to every sample; averaging `R` captures cancels most of it - the noise shrinks by the square root of `R`. The price is `R` times as many captures. In the grid, each 4x increase in averaging let the attack survive about 2x more noise.

#	Question	Answer
6	Why does a small random delay break the single-capture attack? What does averaging do about it, and when does that stop working?	A random shift means identical guesses no longer line up sample by sample, so their difference is large even for wrong guesses - a delay of about 3 samples already breaks a single capture. Because every shift is random and independent, **averaging 4 to 16 captures** blurs them into an average where the right guess still stands out (up to about 8 samples of delay). At 16 samples or more even 16 averages barely help and the attacker must **re-align** the traces first.
7	Name two ways a designer could stop this attack.	Compare the whole password in **constant time** (never stop early); add **random delays** (the attacker then needs re-alignment); **limit wrong attempts** or lock out; hide or mask the check in hardware.

You are done when...

	Check
[ ]	You found the split sample for each position and saw it move **40 samples** per correct character (Step 2).
[ ]	You scored every character and can say why h wins (Step 3).
[ ]	You recovered the whole password and counted the captures: **185** (Step 4).
[ ]	You found how much noise the attack tolerates and how averaging helps (Step 5).
[ ]	You found how big a random delay breaks it, and when averaging stops helping (Step 6).
[ ]	**With a Husky:** you ran Steps 1 to 4 against your own chip (Step 7).

Next: the following SCA101 labs go from *how long* to *how much power*: the **value** of the data being processed changes the power, and that is how AES keys are recovered.

Troubleshooting

Find the message you see, read what it means, try the fix. The **Source** line says how each entry was established.

Setup and the recorded traces

#	What you see	What it means	Fix
E-1	`ModuleNotFoundError: No module named 'cwtraces'`, or the notebook's `ImportError: The 'cwtraces' package is missing ...`	The recorded-trace package is not installed. The standard install steps do not install it. Source: Reproduced	In your ChipWhisperer environment: `python -m pip install cwtraces`. Then **Kernel > Restart** and run the Setup cells again.
E-2	`ValueError: Only guesses up to 5 chars recorded, sorry about that.`	The simulator only knows guesses of up to 5 characters. In Step 3, `KNOWN_PREFIX` plus one more character was too long. Source: Source code	Use a `KNOWN_PREFIX` of at most 4 characters in Step 3 (the simulated password has 5).
E-3	`ValueError: Part of guess (...) not in recorded enumeration list`	The simulator only records the characters a-z, 0-9, space and `\x01`. Source: Source code	Use only those characters in a guess.
E-4	`NameError: name 'get_trace' is not defined` (or `cap_pass_trace`, `score`, ...)	The Setup cells have not been run in this session (for example after a kernel restart). Source: Notebook source code	**Kernel > Restart & Run All**, or run the two Setup cells again.
E-5	`NameError: name 'MODEL_WINDOW'` (or `LIBRARY`, `model_attack`) in Step 6	Step 6 uses the model built in Step 5. Source: Notebook source code	Run the Step 5 cell first (or use **Restart & Run All**).
E-6	The plot or numbers did not change after you edited a cell	Edits only take effect when you run the cell. Source: General Jupyter	Press **Shift+Enter** (or **Ctrl+Enter**) on the cell you edited.

Results that look wrong

#	What you see	What it means	Fix
E-7	Step 4 recovers the wrong password, or only the first few characters	The comparison window ends before the traces split, or there is too much noise for the settings. Source: Reproduced	Use `ATTACK_START, ATTACK_END = 0, 3000`. A window that ends at 500 gets only the first two characters; at 600, four.

#	What you see	What it means	Fix
E-8	A bar chart with several equally tall bars and a margin near 1.0x	The window you chose does not contain the split (Step 3), so every character looks the same. Source: Reproduced	Move the window so it covers sample 433 (and later), for example `440, 600` or the whole trace.
E-9	Step 4 or Step 5 takes a long time	Simulated mode: each capture takes about 0.1 s, so 185 captures take about 16 s and doubling `ATTACK_REPEATS` doubles it. Step 5 builds its model first (about 10 s). Source: Reproduced	Wait; or lower `ATTACK_REPEATS`, `TRIALS` or the length of `NOISE_LEVELS`. On hardware, time one capture with the Step 7 cell.
E-10	Results differ slightly every time you run Steps 3 to 6	The simulator returns a random one of 100 recorded traces per guess, and the models add random noise. Source: Source code	Expected. Margins and counts move a little; the pattern should not.

Hardware (Step 7)

#	What you see	What it means	Fix
E-11	`OSError: Could not find ChipWhisperer` or `Unable to communicate with found ChipWhisperer ...`	The Husky is not visible, or another program already holds it. Source: Source code	**Kernel > Shut Down All Kernels**, close other lab tabs, unplug and replug the Husky. On Linux see the Linux guide section on USB devices.
E-12	`RuntimeError: firmware build failed`	The firmware could not be built (missing compiler, or the notebooks are not inside the `chipwhisperer` folder). Source: Notebook source code	Run Lab 0 check 3. On Linux see the Linux guide section on compilers and firmware builds.
E-13	`OSError: timeout` or `Verify FAILED` while flashing	The wrong target board is on the CW313 (the look-alike iCE40 board), it is not seated, or the Husky is stuck. Source: Husky kit bring-up	Check the board's printed text says SAM4S, reseat it, then **physically unplug and replug the Husky** and run the Setup cell again.
E-14	`Timeout happened during acquisition`, or traces that are flat	The scope never saw the trigger, or the Measure SMA cable is not connected. Source: Notebook source code / Husky kit bring-up	Check the CW313 jumper `JP2` is on T-GPIO4 and the Husky's `Pos` SMA is connected to the CW313 (see the hardware pictures in the Linux guide).

Appendix A - every variable you can edit

Step	Variable	Default	What it does
Setup	`VERSION`	`'SIMULATED'`	`'SIMULATED'` = recorded traces; `'HARDWARE'` = your own chip.
1	`GUESS_A`, `GUESS_B`	`"h"`, `"0"`	The two guesses to overlay.
1	`START, END`, `SMOOTH`	`0, 1000`, `1`	Zoom window and smoothing.
2	`STUDY_PASSWORD`	`"h0px3"`	The password to study.
2	`THRESHOLD`	`0.05`	How big a difference counts as 'split'.
2	`AVERAGES`, `SMOOTH_DIFF`	`10`, `1`	Captures averaged; smoothing of the difference.
3	`KNOWN_PREFIX`	`""`	Characters already found.
3	`WINDOW_START, WINDOW_END`	`0, 3000`	Samples compared.
3	`METRIC`, `REPEATS`	`"sum_abs"`, `1`	Scoring method; captures averaged.
4	`PASSWORD_LENGTH`	`5`	Characters to recover.
4	`ATTACK_START, ATTACK_END`	`0, 3000`	Samples compared.
4	`ATTACK_METRIC`, `ATTACK_REPEATS`	`"sum_abs"`, `1`	Scoring method; captures averaged.
5	`NOISE_LEVELS`, `REPEAT_LEVELS`	see Step 5	Rows and columns of the noise grid.
5	`TRIALS`	`10`	Attacks per cell.
5	`MODEL_WINDOW`, `MODEL_METRIC`	`(0, 3000)`, `"sum_abs"`	Samples compared and scoring in the model.
6	`JITTER_LEVELS`, `JITTER_REPEATS`	see Step 6	Rows and columns of the delay grid.
6	`JITTER_NOISE`, `JITTER_TRIALS`	`0.01`, `5`	Ordinary noise; attacks per cell.
6	`SHOW_JITTER`	`8`	Delay for the example charts.

Appendix B - Glossary

Term	Meaning
password checker	Firmware that compares a guess with the stored password one character at a time and stops at the first difference.
power trace	The recorded power draw of the chip while it runs; here 3000 samples.
split (divergence)	The sample where the traces of a right and a wrong guess stop being the same.
reference	A trace of a guess known to be wrong (here a guess ending in `\x01`), used to compare the others against.
score / metric	One number saying how different two traces are: `sum_abs`, `sum_sq` or `max_abs`.
window	The range of samples used in the comparison.
averaging	Taking several captures of the same guess and averaging them; random noise shrinks by the square root of the number of captures.
noise	Random variation added to every sample.
random delay (jitter)	A defence: the chip waits a random time before the check, so traces are shifted by a different amount each time.
re-alignment	Lining traces up again (for example on a feature) before comparing them.
brute force	Trying every possible password. For 5 characters from 36 that is 36^5 = 60,466,176 guesses.
model	A simplified stand-in built from real recorded traces; good for asking 'what if', not a new measurement.
`cwtraces`	The Python package that holds the recorded traces for the simulated labs.

Notice

This guide accompanies ChipWhisperer course material that references (commercial) training material on ChipWhisperer.io. You can freely execute and use the lab per the open-source license (including in your own courses if you distribute similarly), but you must maintain notice about the source: https://www.ChipWhisperer.io.

Copyright (C) NewAE Technology Inc., 2015-2020. ChipWhisperer is a trademark of NewAE Technology Inc., claimed in all jurisdictions, and registered in at least the United States of America, European Union, and Peoples Republic of China. Tutorials derived from our open-source work must be released under the associated open-source license, and notice of the source must be clearly displayed.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.